

Week 6, Part 2: Interactivity and Navigation

Dr M AOUE

Lecture notes

Objectives. This is the second part of week 6. By the end you will be able to display a list of any length with `ListView.builder`, and move it from your code with a `ScrollController`; build a login screen with a `Form`; choose between `push`, `pushReplacement` and `pushAndRemoveUntil`, and pass data to the next screen through its constructor; show Material, Cupertino and platform-aware dialogs; wait for the value a dialog returns; show modal and persistent bottom sheets; and put all of it together in the complete Stopwatch app.

The **programs** of this part are not printed here: they are in the project **examples**, inside `code/week06` (section 11). Each section quotes only the few lines that matter. Every program starts with a comment that says what it teaches and what result to expect, and automated tests check that it does what is described.

This part follows the chapter *Adding Interactivity and Navigation to Your App*. Its first recipes (state, buttons, the Stopwatch, `ListView`, `TextField`) were already covered in week 5, part 1 (the Stopwatch in three steps) and week 5, part 3 (forms and validation), and are not repeated. Part 1 of this week, *Flutter Routes*, introduced the route stack, `Navigator.push` and `pop`, and constructor parameters. Here we continue from there.

1 UI and UX

Front-end design is often split in two. The **user interface** (UI) is everything on the screen: text, images, colours, panels. The **user experience** (UX) is what happens when the user *acts* on it: interactivity, navigation, animations. If the UI is the “what” of an app, the UX is the “how”. Weeks 4 and 5 were mostly UI and state; this part is about UX: lists that react to the user, several screens, dialogs and bottom sheets.

Throughout the part we finish one app, the **Stopwatch** of week 5: a runner logs in, times laps, and sees a summary when the run stops.

2 Long lists: `ListView.builder`

Week 5 built the lap list with `ListView(children: [...])` and a `collection-for`. That list creates a widget for **every** lap, even the laps far below the screen. With a few laps this does not matter. With thousands of rows (contacts, messages, songs) the app would create thousands of widgets, and the elements and render objects behind them, that nobody sees.

The trick that mobile apps have always used is to build **only what is visible**. A screen can show perhaps seven rows at a time, whatever the length of the list; when a row leaves the screen it can be dropped, and the row that enters is built on demand.

In Flutter this is `ListView.builder`. Instead of a list of widgets, it takes a **function** that builds row number `index` when Flutter needs it:

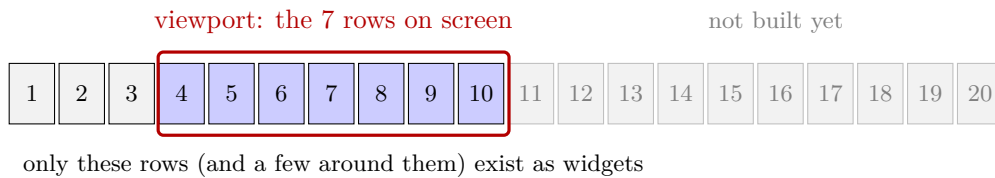


Figure 1: `ListView.builder` creates widgets only for the rows inside the viewport. When the viewport moves, rows that leave are dropped and rows that enter are built.

```

1 Widget _buildLapDisplay() {
2   return Scrollbar(
3     controller: _scrollController,
4     child: ListView.builder(
5       controller: _scrollController,
6       itemExtent: itemHeight,           // every row is 60 pixels tall
7       itemCount: _laps.length,         // how many rows in total
8       itemBuilder: (context, index) {   // builds row number index
9         final milliseconds = _laps[index];
10        return ListTile(
11          contentPadding: const EdgeInsets.symmetric(horizontal: 50),
12          title: Text('Lap ${index + 1}'),
13          trailing: Text(_secondsText(milliseconds)),
14        );
15      },
16    ),
17  );
18 }

```

Field	Meaning
<code>itemBuilder</code>	<code>(context, index) => widget</code> : called by Flutter, with the index it needs, each time a row comes on screen
<code>itemCount</code>	the number of rows. Without it the list is infinite : Flutter asks for <code>index</code> , <code>_laps.length</code> , and <code>_laps[index]</code> throws a <code>RangeError</code> (a test checks it)
<code>itemExtent</code>	a fixed height for every row. Flutter does not have to measure the rows any more, and the position of row n is simply $n \times \text{itemExtent}$
<code>controller</code>	a <code>ScrollController</code> , to read and change the scroll position from your code

`Scrollbar` draws the usual bar that shows where you are in a long list. Because our list has its own controller, the `Scrollbar` receives the **same** controller, so that both follow the same position.

Moving the list from the code: `ScrollController`

A **controller** is an object that lets code *outside* a widget's build method act on that widget. You already know `TextEditingController` (week 5, part 1); `ScrollController` is the same idea for lists. We want every new lap to slide the list to the bottom, so that the newest lap is always visible:

```

1 static const double itemHeight = 60;
2 final ScrollController _scrollController = ScrollController();
3
4 void _lap() {
5   setState(() {
6     _laps.add(_milliseconds);
7     _milliseconds = 0;

```

```

8   });
9   _scrollToEnd();
10  }
11
12  void _scrollToEnd() {
13    if (!_scrollController.hasClients) return; // the list is not on screen
14    final visibleHeight = _scrollController.position.viewportDimension;
15    final end = itemHeight * _laps.length - visibleHeight;
16    if (end <= 0) return; // every lap fits: nothing to do
17    _scrollController.animateTo(
18      end,
19      duration: const Duration(milliseconds: 500),
20      curve: Curves.easeIn, // starts slowly, then speeds up
21    );
22  }

```

- Because every row is `itemHeight` tall, the whole list is `itemHeight * _laps.length` tall, and the last row is fully visible when the list is scrolled to that height *minus* the visible height. This is exact, and known even before Flutter has built the new row: one more benefit of `itemExtent`.
- `animateTo(offset, duration:, curve:)` slides to `offset`; `jumpTo(offset)` goes there at once, without animation.
- Like a `Timer` and a `TextEditingController`, a `ScrollController` holds resources and must be **disposed**:

```

1  @override
2  void dispose() {
3    _timer?.cancel();
4    _scrollController.dispose();
5    super.dispose();
6  }

```

3 A login screen

To know *which* runner we are timing, the app starts with a login screen. Everything in it was covered in week 5, part 3: a `Form` with a `GlobalKey<FormState>`, two `TextFormField`s with their own `TextEditingController`s, a validator for each field, and a button that calls `validate()`. Example 01_login_form puts them together; three details are worth a closer look.

One screen, two faces.

A `bool` in the state chooses which widget tree build returns. `setState` flips it, and the whole body changes:

```

1  body: Center(
2    child: _loggedIn ? _buildSuccess() : _buildLoginForm(),
3  ),

```

This is fine for a small change on one screen. For a different *page*, the next section uses real navigation instead.

The email field.

`keyboardType: TextInputType.emailAddress` shows the keyboard with an “@” key, and the validator checks the text with a regular expression:

```

1  String? _validateEmail(String? text) {
2    if (text == null || text.trim().isEmpty) {
3      return "Enter the runner's email.";

```

```

4 }
5 final regex = RegExp(r'^[^\s]+@[^\s]+\.[^\s]+$');
6 if (!regex.hasMatch(text.trim())) {
7   return 'Enter a valid email';
8 }
9 return null;
10 }

```

Piece	Meaning
^	the start of the text
[^\s]+	one or more characters that are neither “@” nor a space
@	the character “@”
\.	a real dot (a plain . means “any character”)
\$	the end of the text
r'...'	a <i>raw</i> string: \ is an ordinary character in it

The ^ and \$ matter: without them, the pattern only has to match *somewhere* inside the text.

The direction of the test.

validate() returns true when **every** field is valid, so we stop when it returns false:

```

1 void _validate() {
2   if (!_formKey.currentState!.validate()) {
3     return; // at least one field is wrong: its error is already shown
4   }
5   setState(() {
6     _loggedIn = true;
7     _name = _nameController.text.trim();
8   });
9 }

```

The printed text first writes `if (form?.validate() ?? false) { return; }.` This returns when the form is *valid*, and logs in when it is *not*: exactly the opposite of what we want. A test runs that code on an empty form: the error “Required” appears *and* the user is logged in. The chapter’s later version, `if (form?.validate() == false) return;`, is correct.

4 Navigating to the next screen

Week 6, part 1 showed that `MaterialApp` keeps the screens, called **routes**, in a **stack**: `Navigator.push` puts a route on top, `Navigator.pop` removes it. After a login there is a problem: with `push`, the login screen stays *below* the stopwatch, and the Back button returns to it. In most apps, once you have logged in, the login screen should no longer be reachable.

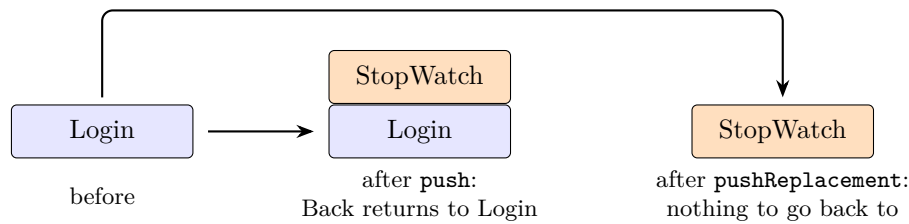


Figure 2: `push` adds the new route on top of the current one; `pushReplacement` replaces the current route by the new one.

Method	Effect on the stack
<code>push(route)</code>	adds <code>route</code> on top; Back returns to the current screen
<code>pushReplacement(route)</code>	removes the current route and adds <code>route</code> : the current screen cannot be reached any more
<code>pushAndRemoveUntil(route, test)</code>	adds <code>route</code> , and removes the routes below it until <code>test(route)</code> returns <code>true</code> ; with <code>(r) => false</code> it removes all of them
<code>pop([result])</code>	removes the top route, optionally giving a <code>result</code> to whoever pushed it (section 6)
<code>canPop()</code>	<code>true</code> if there is a route below the current one

So the login screen *replaces* itself by the stopwatch:

```

1 void _validate() {
2   if (!_formKey.currentState!.validate()) {
3     return;
4   }
5   final name = _nameController.text.trim();
6   final email = _emailController.text.trim();
7   Navigator.of(context).pushReplacement(
8     MaterialPageRoute(builder: (_) => Stopwatch(name: name, email: email)),
9   );
10 }
  
```

- The data goes to the next screen through its **constructor**, with typed, required parameters, as in part 1 (“constructor parameters”). In the `State` class the widget’s fields are read through `widget.title: Text(widget.name)`.
- `MaterialPageRoute` takes a `builder`, not a widget, like `ListView.builder`: Flutter calls it only when the route is really shown. Its parameter is a `BuildContext` that we do not use, so we name it `_`.
- The `AppBar` draws the back arrow **by itself** when `Navigator.of(context).canPop()` is `true`. After `pushReplacement` there is no arrow, and we wrote no code for it.
- `MaterialPageRoute` chooses an animation that suits the platform: on iOS the new screen slides in from the right. On Android, the current Flutter uses a fade and zoom animation that follows the system’s “predictive back” gesture (we checked the default in the framework source); older versions used other animations.

Example 02_push_vs_replacement shows the difference side by side: the same *Welcome* screen opened with **PUSH** has a back arrow and says “Routes below this one: yes”; opened with **PUSH REPLACEMENT** it has no arrow and says “no”. Its **LOG OUT** button uses `pushAndRemoveUntil` to start again from an empty stack:

```

1 Navigator.of(context).pushAndRemoveUntil(
2   MaterialPageRoute(builder: (_) => const LoginScreen()),
3   (route) => false,    // remove every route below
4 );

```

5 Dialogs

A dialog (a “popup”) gives the user a message that needs their attention: an error, a warning, a question that must be answered before going on. Because a dialog interrupts the user, show one only when it is really necessary.

A dialog is not placed in the widget tree like a `Text`. It is shown by calling a **function**, which, like `MaterialPageRoute`, takes a **builder**:

```

1 showDialog<void>(
2   context: context,
3   builder: (context) {
4     return AlertDialog(
5       title: const Text('Run completed!'),
6       content: const Text('Total run time is 12.3 seconds.'),
7       actions: [
8         TextButton(
9           onPressed: () => Navigator.of(context).pop(),
10          child: const Text('Close'),
11        ),
12      ],
13    );
14  },
15 );

```

A dialog **is a route**. `showDialog` pushes it on the same `Navigator` stack as the screens; the only difference with a `MaterialPageRoute` is the way it appears (on top of the page, which is darkened, instead of replacing it). That is why it is closed with `Navigator.of(context).pop()`.

Material and Cupertino

	Material (Android look)	Cupertino (iOS look)
Function	<code>showDialog</code>	<code>showCupertinoDialog</code>
Dialog	<code>AlertDialog</code>	<code>CupertinoAlertDialog</code>
Actions	<code>TextButton</code>	<code>CupertinoDialogAction</code>
Tap outside	closes the dialog (<code>barrierDismissible</code> is true)	does nothing: the user must press an action

The difference for a tap outside the dialog is checked by a test.

A platform-aware dialog

The chapter writes a small class, `PlatformAlert`, that hides the choice from the rest of the app. Its `show` method reads the platform from the theme and shows the dialog of the right family:

```

1 class PlatformAlert {
2   const PlatformAlert({required this.title, required this.message});
3
4   final String title;
5   final String message;
6
7   Future<void> show(BuildContext context) {

```

```

8   final platform = Theme.of(context).platform;
9   if (platform == TargetPlatform.iOS || platform == TargetPlatform.macOS) {
10    return _showCupertinoAlert(context); // showCupertinoDialog +
    CupertinoAlertDialog
11  }
12  return _showMaterialAlert(context); // showDialog + AlertDialog
13 }
14 ...
15 }

```

The rest of the app only writes one line. In the stopwatch, `_stopTimer` shows the total time of the run, the finished laps *plus* the lap that was running:

```

1 final totalRuntime = _laps.fold(_milliseconds, (total, lap) => total + lap);
2 PlatformAlert(
3   title: 'Run completed!',
4   message: 'Total run time is ${_secondsText(totalRuntime)}.',
5 ).show(context);

```

`fold(start, f)` goes through the list and combines its values two by two: it starts from `_milliseconds`, then adds each lap.

Built into Flutter: `AlertDialog.adaptive`.

Flutter now has the same idea built in: `showAdaptiveDialog` with `AlertDialog.adaptive(...)` gives an `AlertDialog` on Android, Windows, Linux and the web, and a `CupertinoAlertDialog` on iOS and macOS. Its actions are ordinary `TextButtons` in both cases. Writing `PlatformAlert` yourself is still a good exercise: it is the pattern you will use for any widget that must look different on each platform.

Seeing the iOS dialog without an iPhone.

`Theme.of(context).platform` is the platform *of the theme*, which is the real one by default but can be changed. Example 03_platform_alert has a switch, “Pretend to be iOS”, that rebuilds the app with `ThemeData(platform: TargetPlatform.iOS)`; on Windows, Android or the web, **PLATFORM ALERT** and **ADAPTIVE DIALOG** then show the iOS dialog.

6 Dialogs that return a value

`showDialog<T>` returns a `Future<T?>`. The value is the one given to `Navigator.pop` when the dialog is closed, so the code that opened the dialog can **wait** for the user’s answer with `await` (week 1: futures and `async/await`):

```

1 Future<void> _clearLaps() async {
2   final ok = await showDialog<bool>(
3     context: context,
4     builder: (context) => AlertDialog(
5       title: const Text('Clear all laps?'),
6       content: const Text('This cannot be undone.'),
7       actions: [
8         TextButton(onPressed: () => Navigator.of(context).pop(false), child:
9         const Text('Cancel')),
10        TextButton(onPressed: () => Navigator.of(context).pop(true), child:
11        const Text('Clear')),
12      ],
13    ),
14  );
15  if (ok != true || !mounted) return; // false, or null (tap outside)
16  setState(() => _laps.clear());
17 }

```

- The answer can be `null`: the user can close a Material dialog by tapping outside it or with the Back button, and then nobody called `pop` with a value. Test `ok != true`, not `!ok`, which does not compile on a `bool`? (*verified error: A nullable expression can't be used as a condition*).
- After an `await`, time has passed: the user may have left the screen in the meantime. Before using `context` or `setState` again, check that the `State` is still in the tree with `if (!mounted) return;`. In a `StatelessWidget`, or with a `context` received as a parameter, the check is `if (!context.mounted) return;`.
- A `SimpleDialog` shows a list of choices. Each `SimpleDialogOption` pops the dialog with its own value; example `04_dialog_result` uses it to choose a distance:

```

1 final chosen = await showDialog<int>(
2   context: context,
3   builder: (context) => SimpleDialog(
4     title: const Text('Distance of one lap'),
5     children: [
6       for (final metres in [100, 200, 400, 800])
7         SimpleDialogOption(
8           onPressed: () => Navigator.of(context).pop(metres),
9           child: Text('$metres m'),
10        ),
11     ],
12   ),
13 );
14 if (chosen == null || !mounted) return; // closed without a choice
15 setState(() => _distance = chosen);

```

The same idea works for screens: `await Navigator.of(context).push(...)` gives the value that the pushed screen passes to `pop`.

7 Bottom sheets

A bottom sheet is a panel that slides up from the bottom of the screen. It is less intrusive than a dialog, and is well suited to a short list of options or a summary. There are two kinds:

	Modal	Persistent
Shown with	<code>showModalBottomSheet(context:, builder:)</code>	<code>Scaffold.of(context).showBottomSheet(builder)</code>
The page above	darkened and blocked, as with a dialog	still works: you can scroll and tap
It is	a route: closed with <code>Navigator.pop(context, value)</code>	part of the <code>Scaffold</code>
Returns	a <code>Future<T?></code> with the value given to <code>pop</code> (null if the user taps the dark area or drags the sheet down)	a <code>PersistentBottomSheetController</code> : <code>close()</code> closes it, <code>closed</code> completes when it is gone

A modal sheet that returns a choice

Example `05_bottom_sheets` sorts the laps. The modal sheet lists the three orders and returns the one the user taps, exactly like the `SimpleDialog` above:

```

1 final chosen = await showModalBottomSheet<LapOrder>(
2   context: context,
3   builder: (context) => SafeArea(

```

```

4      child: Column(
5        mainAxisAlignment: MainAxisAlignment.min,    // only as tall as its content
6        children: [
7          for (final order in LapOrder.values)
8            ListTile(
9              title: Text(order.label),
10             onTap: () => Navigator.of(context).pop(order),
11            ),
12          ],
13        ),
14      ),
15    );
16    if (chosen == null || !mounted) return;
17    setState(() => _order = chosen);

```

`mainAxisAlignment: MainAxisAlignment.min` makes the `Column` only as tall as its children; otherwise the sheet would take the maximum height it is allowed. `SafeArea` keeps the content away from the system bars and the rounded corners of the phone.

A persistent sheet, and the context problem

`Scaffold.of(context)` looks **up** the tree, from `context`, for the nearest `Scaffold`. This is the same mechanism as `Theme.of` and `Form.of` (week 5, part 3), and it has the same trap.

The context of the build method that *creates* the `Scaffold` is *above* it. `Scaffold.of(context)` there fails with “*Scaffold.of() called with a context that does not contain a Scaffold*” (a test checks it). The fix is a `Builder`: its builder function receives a new context, for the point where the `Builder` sits, which is below the `Scaffold`.

```

1  return Scaffold(
2    ...
3    bottomNavigationBar: Builder(
4      builder: (scaffoldContext) => ElevatedButton(    // a context BELOW the
5        Scaffold
6        onPressed: () => _showSummary(scaffoldContext),
7        child: const Text('Summary'),
8      ),
9    ),
10 );
11 ...
12 void _showSummary(BuildContext scaffoldContext) {
13   final controller = Scaffold.of(scaffoldContext).showBottomSheet(
14     (context) => Container( ... 'Run completed - 5 laps, total 301.5 s' ... ),
15   );
16   _closeTimer = Timer(const Duration(seconds: 5), controller.close);
17   controller.closed.then((_) {
18     _closeTimer?.cancel();
19     if (mounted) setState(() => _summarySheet = null);
20   });
21 }

```

- The sheet closes itself after five seconds: a one-shot `Timer` calls `controller.close`. Like every `Timer`, it is cancelled in `dispose`.
- `controller.closed` is a `Future` that completes when the sheet is gone, whether it was closed by the timer or by its own close button. We use it to enable the **SUMMARY** button again.
- While the sheet is open, the page above still works: a test opens the summary, then sorts the list through the modal sheet, and checks that both worked.

8 The complete Stopwatch app

Example 06_stopwatch_app puts the whole part together, in four files, like the sum app of week 5:

File	Role
main.dart	the root: a <code>MaterialApp</code> whose <code>home</code> is the login screen
login_screen.dart	the Form; when it is valid, <code>pushReplacement</code> opens the stopwatch and passes the name and email
stopwatch.dart	the laps (<code>ListView.builder</code> , <code>ScrollController</code>); a <code>PlatformAlert</code> when it stops; a modal bottom sheet with the details of a lap when it is tapped; Log out with a confirmation dialog, then <code>pushReplacement</code> back to the login
platform_alert.dart	the <code>PlatformAlert</code> class

The login screen imports `stopwatch.dart` to open the stopwatch, and the stopwatch imports `login_screen.dart` to come back on **Log out**: in Dart, two files may import each other. The Log out button combines two sections of this part, a dialog that returns a value and a replacement route:

```

1 Future<void> _logout() async {
2   final ok = await showDialog<bool>( ... 'Log out?' ... Cancel: pop(false), Log
    out: pop(true) ... );
3   if (ok != true || !mounted) return;
4   Navigator.of(context).pushReplacement(
5     MaterialPageRoute(builder: (_) => const LoginScreen()),
6   );
7 }

```

9 What we changed from the printed text

The chapter was written for an older version of Flutter. The programs of this course use the current API, and we **verified** each correction below, either by compiling the old form with `flutter analyze` and reading the real message, with an automated test, or in the framework source:

In the printed text	In our programs, and why
<code>textTheme.headline5</code>	<code>textTheme.headlineSmall</code> . <i>Verified error: The getter 'headline5' isn't defined for the type 'TextTheme'.</i> The old names were removed from Flutter.
<code>ElevatedButton.styleFrom(primary: Colors.green)</code>	<code>styleFrom(backgroundColor: ..., foregroundColor: ...)</code> . <i>Verified error: The named parameter 'primary' isn't defined.</i>
<code>ButtonStyle(backgroundColor: MaterialStateProperty.all(...))</code>	still compiles, but <i>deprecated: 'MaterialStateProperty' is deprecated ... Use WidgetStateProperty instead.</i> We use the shorter <code>styleFrom</code> .
<code>if (form?.validate() ?? false) { return; }</code>	<code>if (!_formKey.currentState!.validate()) return;</code> . The printed test is inverted: a test shows that it logs in with an <i>invalid</i> form.

<code>RegExp('[^@]+@[^.]+\.\.\.')</code>	<code>RegExp(r'[^@\s]+@[^\s]+\.[^\s]+\$')</code> . In the printed pattern <code>..</code> means “any two characters”, not a dot, and nothing is anchored: a test shows that it accepts <code>a@bcd</code> and <code>lina @ mail com</code> .
<code>late Timer timer;</code> with <code>initState</code> removed	<code>Timer? _timer;</code> and <code>_timer?.cancel()</code> . If <code>Start</code> is never pressed, <code>dispose</code> reads a <code>late</code> variable that was never assigned and throws a <code>LateInitializationError</code> (verified in week 5, part 1).
<code>seconds++</code> on a line of its own	<code>seconds++;</code> <i>Verified error: Expected to find ';'.</i>
<code>class StopwatchState</code> in step 6, <code>_StopWatchState</code> elsewhere	one name, <code>_StopWatchState</code> , the one returned by <code>createState</code> . Two different names would declare a second class.
<code>color:</code> <code>Theme.of(context).primaryColor</code> with white text	<code>colorScheme.primary</code> with <code>colorScheme.onPrimary</code> : with Material 3, <code>primaryColor</code> is not the colour of the scheme, and <code>onPrimary</code> is the text colour designed to be readable on <code>primary</code> .
<code>CupertinoButton</code> in the actions of a <code>CupertinoAlertDialog</code>	<code>CupertinoDialogAction</code> , the widget made for those actions (it compiles either way, but only the second looks like an iOS dialog).
“the form’s <code>validateAndSubmit</code> method”	there is no such method: <code>FormState</code> has <code>validate()</code> , <code>save()</code> and <code>reset()</code> .
“ <code>onChanged</code> is on the core <code>TextField</code> (not <code>TextFormField</code>)”	<code>TextFormField</code> has an <code>onChanged</code> too (we checked the framework source).
<code>MaterialPageRoute</code> : “on Android, it will pop onto the screen from the bottom”	the current default on Android is a fade and zoom animation for predictive back (we checked the framework source).

10 Common errors

Message or symptom	Cause	Fix
<code>RangeError (index)</code> while scrolling	<code>ListView.builder</code> without <code>itemCount</code> : the list is infinite	<code>add itemCount: _laps.length</code>
<i>Vertical viewport was given unbounded height</i>	a <code>ListView</code> directly in a <code>Column</code> (week 5, part 1)	wrap it in <code>Expanded</code>
<i>A <code>ScrollController</code> was used after being disposed</i>	the controller is disposed and then used again	dispose it exactly once, in the <code>State</code> ’s <code>dispose</code>
The Back button returns to the login screen	the stopwatch was opened with <code>push</code>	use <code>pushReplacement</code>
The dialog never closes	its button does not call <code>Navigator.pop</code>	a dialog is a route: pop it
<i><code>Scaffold.of()</code> called with a context that does not contain a <code>Scaffold</code></i>	the <code>context</code> belongs to the widget that <i>creates</i> the <code>Scaffold</code>	use a <code>Builder</code> below the <code>Scaffold</code>

<i>A nullable expression can't be used as a condition, for if (!ok) on the answer of a dialog</i>	the answer is a bool? (verified)	test ok != true or ok == true
<i>Don't use 'BuildContext's across async gaps (analyzer), or a crash after closing a screen</i>	context or setState used after an await without checking	if (!mounted) return; after the await
Always the Material dialog, even on an iPhone	showDialog with an AlertDialog used directly	use PlatformAlert or AlertDialog.adaptive

11 The programs

The code of this part is **not** printed in this document. It is in the folder `code/week06/examples`. See a *Running the Flutter apps in VS Code* guide (week 4 or 5) to prepare the tools; then, from the folder `examples`:

```
flutter pub get
flutter test
flutter create .                (one time only)
flutter run                     (the complete Stopwatch app)
flutter run -t lib/03_platform_alert.dart
```

File	What you see and try
00_lap_list_builder	the week 5 stopwatch; press LAP many times: the list scrolls by itself to the newest lap, and shows a scrollbar
01_login_form	CONTINUE with empty fields shows two errors; lina@mail is refused; a valid form is replaced by "Hi Lina"
02_push_vs_replacement	PUSH opens a screen with a back arrow, PUSH REPLACEMENT without; LOG OUT returns to a fresh login
03_platform_alert	four dialogs; turn on "Pretend to be iOS" and the platform-aware and adaptive ones change to the iOS style
04_dialog_result	choose a distance in a SimpleDialog ; confirm or cancel "Clear all laps?"
05_bottom_sheets	SORT opens a modal sheet that sorts the laps; SUMMARY opens a persistent sheet that closes after 5 seconds
06_stopwatch_app/	the complete app: log in, time laps, stop (dialog), tap a lap (bottom sheet), log out (confirmation)

12 Exercises

1. In `00_lap_list_builder`, remove `itemCount`, run, and press **LAP**: read the error in the debug console.
2. In `00_lap_list_builder`, replace `animateTo` by `jumpTo(end)`: what changes on the screen?
3. In `00_lap_list_builder`, change `itemHeight` to 40 and check that the newest lap is still fully visible. Why does it still work?
4. In `01_login_form`, add a third field, "Age", that accepts only a whole number between 10 and 99 (use `int.tryParse`).

5. In `02_push_vs_replacement`, replace `pushAndRemoveUntil` in **LOG OUT** by a plain `push`. Press **PUSH**, then **LOG OUT**: how many times must you press Back to leave?
6. In `03_platform_alert`, add a second action, “Share”, to the `PlatformAlert`; it should also close the dialog.
7. In `03_platform_alert`, give `showDialog` the argument `barrierDismissible: false`: can a tap outside still close the Material dialog?
8. In `04_dialog_result`, write `if (!ok) return;` instead of `if (ok != true ...)` and read the analyzer’s error.
9. In `05_bottom_sheets`, call `Scaffold.of(context)` with the `context` of the `build` method instead of `scaffoldContext`, and read the error.
10. In `05_bottom_sheets`, add a fourth order to `LapOrder`: the lap numbers in reverse (the last lap first).
11. In `06_stopwatch_app`, show the runner’s email under the name in the app bar (hint: `AppBar.title` can be a `Column`).
12. In `06_stopwatch_app`, make **Stop** show a persistent bottom sheet with the total time instead of the dialog, closing after 5 seconds, as in example 05.

13 Summary

Idea	In one sentence
<code>ListView.builder</code>	builds only the visible rows, on demand, with <code>itemBuilder</code> ; always give it <code>itemCount</code>
<code>itemExtent</code>	a fixed row height: faster layout, and the position of row n is $n \times \text{itemExtent}$
<code>ScrollController</code>	moves a list from code (<code>animateTo</code> , <code>jumpTo</code>); dispose it in <code>dispose</code>
<code>Scrollbar</code>	shows the position in a long list; give it the list’s controller
<code>pushReplacement</code>	replaces the current route: the user cannot come back to it
<code>pushAndRemoveUntil</code>	pushes a route and removes the routes below it (all of them with <code>(r) => false</code>)
<code>widget.name</code>	how the <code>State</code> reads the data passed to its widget’s constructor
<code>showDialog</code>	shows a dialog, which is a route: close it with <code>Navigator.pop</code>
<code>PlatformAlert</code>	shows an <code>AlertDialog</code> or a <code>CupertinoAlertDialog</code> depending on <code>Theme.of(context).platform</code> ; <code>AlertDialog.adaptive</code> does the same
<code>await showDialog<T></code>	gives the value passed to <code>pop</code> , or <code>null</code> ; check <code>mounted</code> after the <code>await</code>
<code>showModalBottomSheet</code>	a sheet that blocks the page and returns a value, like a dialog
<code>showBottomSheet</code>	a persistent sheet of the <code>Scaffold</code> ; <code>Scaffold.of</code> needs a context below the <code>Scaffold (Builder)</code>

This document is maintained by Dr. Mohamad AOUE

Last updated: 2026-10-05